Assignment for the course
Automated Planning Theory and Practice
Academic Year 2023-2024

Marco Roveri
marco.roveri@unitn.it

1 Introduction

The aim of this assignment is twofold. First to model planning problems in PDDL/HDDL to then invoke a state of the art planner as those provided by `planutils` [5] or manually compiled (e.g., `fast downward` [2], or `optic` [1]). Second, to see how a temporal planning model could be integrated within a robotic setting leveraging the `PlanSys2` [4] infrastructure discussed in the lectures and available at https://github.com/PlanSys2/ros2_planning_system.

This assignment can be performed alone or in group of *at most two* students.

This year, I propose two scenarios. The first is inspired by the *health care* domain. The second is *free choice*, namely the student is asked to propose a scenario that is not related to the health care domain. The student is free to choose the scenario, and she/he must provide a brief description of the scenario, the initial state, the goal, and the actions that can be performed. The student must also provide a brief justification of the choice of the scenario.

2 The health care scenario

Let us consider a scenario inspired by the health care domain where there are a number of units within a unique healthcare facility each dedicated to providing focused medical care and services. Each unit is at given known location, to provide respective medical treatments. To submit the treatments, they need medical devices and drugs to be at their premises. The objective of the planning system is to orchestrate the activities of a set of different robotic agents to deliver boxes containing needed medical devices and drugs to each unit. Moreover, to also accompany patients to the desired unit. The robotic agents can move between the different units, pick up boxes, deliver, and accompany patients.
Let's consider these assumptions:

- Each medical unit is at a specific location.
- Each box is initially at a specific location (e.g., the warehouse of the health care facility) and can be filled with a specific content such as *scalpel* or *tongue depressor* or *aspirine*, if empty. Box contents shall be modeled in a generic way, so that new kind of contents can easily be introduced in the problem instance (i.e., we shall be able to reason on the content of each box).
- Each medical unit has or does not have a box with a specific content. That is, you must keep track of whether they have e.g., scalpels or not, whether they have tongue depressor or not, whether they have aspirine or not, and so on.
- There can be more than one medical unit at any given location, and therefore it isn't sufficient to keep track of where a box is in order to know which medical units have been given boxes!
- Each robotic agent can:
  - fill a box with a content, if the box is empty and the content to add in the box, the box and the robotic agent are at the same location;
  - empty a box by leaving the content to the current location and given medical unit, causing the medical unit to then have the content;
  - pick up a single box and load it on the robotic agent, if it is at the same location as the box;
  - move to another location, considering that if the robotic agent is loaded with a box, then also the box moves with the agent itself, and if the agent is not loaded with a box, then only the agent moves to the new location;
  - deliver a box to a specific medical unit who is at the same location.
- The robotic agents can move only between connected locations as typical of an healthcare facility (there is a "roadmap" with specific connections that must be taken).
- Since we want to be able to expand this domain for multiple robotic agents in the future, we expect to use a *separate type* for robotic agents, which currently happens to have a single member in all problem instances.
- There are patients that need to be accompanied to the desired medical unit. The robotic agents can accompany patients to the desired unit.
- All the patients are at the entrance (a specific location of the healthcare facility).

- Only specific robotic agents can accompany patients. And they can accompany only one patient at a time, and the robots for accompanying patients are disjoint from the other robots that can carry boxes.

2.1 The Problems

The assignment is structured in 5 sub problems to be solved in sequence where the second builds on the first, both the third and the fourth build on the second, the fifth builds on the fourth.

2.1.1 Problem 1

Initial condition

- Initially all boxes are located at a single location that we may call the `central_warehouse`.
- All the contents to load in the boxes are initially located at the `central_warehouse`.
- There are no medical unit at the `central_warehouse`.
- A single robotic agent allowed to carry boxes is located at the `central_warehouse` to deliver boxes.
- A single robotic agent allowed to accompany patients is initially located at the entrance.

There are no particular restrictions on the number of boxes available, and constraints on reusing boxes! These are design modeling choices left unspecified and that each student shall consider and specify in her/his solution. There are no restrictions on the number of patients to be accompanied.

Goal The goal should be that:

- certain medical units have certain supplies;
- some medical units might not need supply;
- some medical units might need several supplies;
- some patients need to reach some medical unit.

This means that the robotic agent has to deliver to needing medical unit some or all of the boxes and content initially located at the `central_warehouse`, and leave the content by removing the content from the box (removing a content from the box causes the medical unit at the same location to have the unloaded content).

Remarks medical unit don't care exactly which content they get, so the goal should not be (for example) that `MedicalUnit1` has `scissor1`, and `scalpel5`, merely that `MedicalUnit1` has a `scissor` and a `scalpel`.

2.1.2 Problem 2

We leverage the scenario in Section 2.1.1 with the following extensions, where we will have to consider an alternative way of moving boxes (e.g., drones or different kind of robots), and the fact that the robotic agents have a carrier to load the boxes.

- each robotic agent has a carrier with a maximum load capacity (that might be different from each agent);
- the robotic agents can load boxes in the carrier up to the carrier maximum capacity;
- the robotic agent and the boxes to be loaded in the robotic agent carrier shall all be at the same location;
- the robotic agent can move the carrier to a location where there are medical units needing supplies;
- the robotic agent can unload one or more box from the carrier to a location where it is;
- the robotic agent may continue moving the carrier to another location, unloading additional boxes, and so on, and does not have to return to the `central_warehouse` until after all boxes on the carrier have been delivered;
- though a single carrier is needed for the single robotic agent, there should still be a separate type for carriers;
- for each robotic agent we need to count and keep track of i) which boxes are on each carrier; ii) how many boxes there are in total on each carrier, so that carriers cannot be overloaded.
- the capacity of the carriers should be problem specific. Thus, it should be defined in the problem file.

It might be the case additional actions, or modifications of the previously defined actions are needed to model loading/unloading/moving of the carrier (one can either keep previous actions and add new ones that operates on carriers, or modify the previous actions to operate on carriers).

The initial condition and the goal are the same as the problem discussed in Section 2.1.1.

Initial condition

- Initially all boxes are located at a single location that we may call the `central_warehouse`.
- All the contents to load in the boxes are initially located at the `central_warehouse`.
- There are no medical units needing supplies at the `central_warehouse`.
- The robotic agents are located at the `central_warehouse` to deliver boxes.
- Each robotic agent is initially empty.
- Fix the capacity of each robotic agent to be a value greater than 1.
- A single robotic agent allowed to accompany patients is initially located at the entrance.

Goal The goal should be that:

- certain medical units have certain supplies (e.g., bolt, tool, valve);
- some medical units might not need supplies;
- some medical units might need more than one supplies;
- some patients need to reach some medical unit.

2.1.3 Problem 3

We leverage the scenario in Section 2.1.2, but in this case we need to address the problem with hierarchical task networks (HTN). To this extent, it is suggested to keep the same actions as per the solution proposed for Problem 2.1.2, the same initial condition and the same goal (encoded as an HTN). However, here the candidate shall introduce `:tasks` and `:methods` following approaches similar to those discussed in the Lab session on HTN.

2.1.4 Problem 4

We leverage the scenario in Section 2.1.2 with the following extensions.

- Convert the domain defined in Section 2.1.2 to use durative actions. Choose arbitrary (but reasonable durations) for the different actions.
- Consider the possibility to have actions to be executed in parallel when this would be possible in reality. For example, a robotic agent cannot pick up several boxes at the same time, or pick up a box and if it is a drone fly to a destination at the same time.

The initial condition and the goal are the same as the problem discussed in Section 2.1.2.

2.1.5 Problem 5

The final problem consists in implementing within the `PlanSys2` the last problem resulting as outcome of Section 2.1.4 using fake actions as discussed in the tutorials of `PlanSys2` available at [3]. To facilitate the executions, if needed the candidate can enforce that no actions execute in parallel.

3 Free choice scenario

For the free choice scenario, the student is asked to propose a scenario different from the one proposed in the health care domain. The student is free to choose the scenario, and she/he must provide a brief description of the scenario, the initial state, the goal, and the actions that can be performed. The student must also provide a brief justification of the choice of the scenario. The scenario must be complex enough to require the use of HTN, and must follow the same structure of the health care scenario (i.e. it must be composed of steps that build on each other), and in the last step it must be shown a simple deployment in PlanSys2.

4 Deliverables

The deliverable shall consist of a unique archive containing at least the following contents:

- The PDDL/HDDL files (domain and problems) for each of the problems discussed in Sections 2.1.1, 2.1.2, 2.1.3, and 2.1.4 and similar contents for the free choice scenario, possibly organized in folders. All the PDDL/HDDL files shall be valid ones, and shall be parsable correctly by at least one planner (it shall be specified which one, and discussed why the planner has been chosen). The options used to run the chosen planner shall be specified.
- A folder containing all the code to execute the `PlanSys2` problem as discussed in Section 2.1.5 or of the equivalent for the free choice scenario.
- A professional report in PDF describing: (i) a brief description summarizing the understanding of the problems for the health care scenario, or for the free choice scenario a description of the scenario, the initial state, the goal, and the actions that can be performed, and a brief justification of the choice of the scenario with assumptions if any; (ii) the approach followed, justifying and documentin all the design choices (e.g., predicates, actions, ...); (iii) possible additional assumptions (for the part left underspecifed), and in case of deviation from the specification in this document a clear justification. Finally, the document shall include evidence of the attempts to solve the PDDL problems formulated, and running the final version within `PlanSys2` (in form of screenshots and/or output generated by the planner).
  - The report shall also discuss the content of the archive and how it has been organized.
  - The report shall also contain a critical discussion of the results obtained.
  - It would be appreciated experimenting with different solvers and/or options (e.g., different heuristics), discuss posssible differences in the generated solutions, and also some scalability analysis (e.g., considering different initial states, size of the problem, number of agents, number of locations).

The submission shall be performed only through the web form available at the following URL:

Submission Form

`https://forms.gle/PwiWPogfWgo7rfM46`

Report structure:
1- Intoduction
2- Understanding the proble: describe witho your words the understanding of the problem and assumptions to solve the problem. Justify and disambiguous the things
3 - Eval: different planners print the time, n° actions and so on and evaluate the differences. Also when you scale the number of objects

Really detailed report

References

[1] J. Benton, Amanda Jane Coles, and Andrew Coles. Temporal planning with preferences and time-dependent continuous costs. In Lee McCluskey, Brian Charles Williams, José Reinaldo Silva, and Blai Bonet, editors, *Proceedings of the Twenty-Second International Conference on Automated Planning and Scheduling, ICAPS 2012, Atibaia, São Paulo, Brazil, June 25-19, 2012*. AAAI, 2012. URL http://www.aaai.org/ocs/index.php/ICAPS/ICAPS12/paper/view/4699. (page 1)

[2] Malte Helmert. The fast downward planning system. *J. Artif. Intell. Res.*, 26:191–246, 2006. (page 1)

[3] Francisco Martín and colleagues. PlanSys2 Tutorials, 2022. https://plansys2.github.io/tutorials/index.html. (page 3)

[4] Francisco Martín, Jonatan Ginés, Francisco J. Rodríguez, and Vicente Matellán. PlanSys2: A Planning System Framework for ROS2. In *IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS 2021, Prague, Czech Republic, September 27 - October 1, 2021*. IEEE, 2021. (page 1)

[5] Cristian Muise and other contributors. PLANUTILS, 2021. General library for setting up linux-based environments for developing, running, and evaluating planners. https://github.com/AI-Planning/planutils. (page 1)